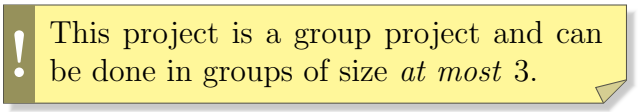
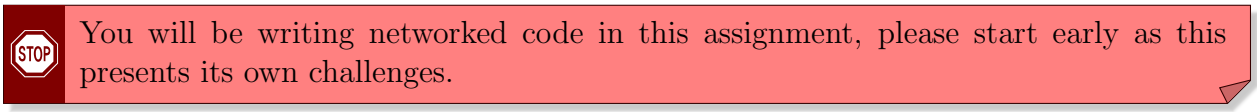
Network Security	**Programming Project 3**
CSC3055 Spring 2025 (Kissel)	**Due Date: Friday March 28, 2025**

Code should be stored in a private repository on GitHub under your Merrimack account. Be sure to use the source control system properly (e.g., multiple commits over the course of the project). The commit history will be used to assess individual contributions in a group project. Remember to test your solution thoroughly. Code that does not work correctly will loose credit. ***Code that does not compile will receive 0 points of credit.*** To submit your project please add me as a collaborator, my username is kisselz@merrimack.edu. Good luck!



In this project you will construct a *single sign-on* (SSO) service that is similar to Kerberos. This requires you to construct a key distribution center (KDC), a client, and a service. The client will authenticate itself to the KDC using CHAP (RFC 1994). Once authenticated, the client may request a ticket to communicate with a service known to the KDC. In this case your only service will be a simple echo service that returns to you the uppercase form of the message it receives. Of course, much more interesting services are possible.



As mentioned you must write three applications for this project.

1. A client application that may either have a command line interface or GUI.
2. A KDC server which *must* be a command line application. Services on your machine are never graphical.
3. An echo service which *must* be a command line application. It echoes back the uppercase form of any string it receives.

I encourage you and your group to participate in the construction of all applications since their operations are joined by the protocols they utilize. Whenever passwords are required to be entered, which is required by the client, you should protect against *shoulder surfing attacks*.

Client Application

The client application should prompt the user for their user name and password. This password will be used as the secret in the CHAP protocol as well as used to derive the root key shared by the client and server. You should not store this password anywhere on the client machine.

The client will participate in both of the protocols:

- The session key protocol (consists of a CHAP authentication phase then a request for a session key and associated ticket).
- The service communication protocol (consists of a handshake phase and communication phase).

These protocols should be run in series such that after the client has concluded interacting with the KDC, it should begin interacting with the service. When interacting with the service you should allow the user to type the message they want echoed back, in some way.

In order to get host information the client should read a hosts file, which is a JSON file that consists of a hosts file object with an array with key `hosts`. Each entry of the array is a host entry object that consists of three fields:

- An field with key `host-name` that represents the host name as string.
- An field with key `address` that represents the address of the host (as a string).
- A field with key `port` that represents the port number for the host.

By default your hosts file *must* contain an `kdcd` host which has the host information for the KDC. As a note, the port and host name information for the KDC and any service should match the associated configuration files for the KDC and service (described below). For reference I have attached a `hosts.json` file to the assignment.



Support for working with the hosts file is available in `merrimackutil.net.hostdb`. Please download version 2025.2 to get access to this support. The `HostsDatabase` class allows you to interact with the hosts database (load and query). See the API documentation on `https://cs.merrimack.edu/merrimackutil/` for more detail.

Please be make sure that the `hosts.json` file is read without user intervention.

KDC Server (Daemon)

The KDC server should read all of its information from a JSON configuration file and then use this information to startup and enter an infinite loop waiting on a new socket connection.

Servers are normally threaded and your server should be as well. Using the thread-pool pattern (`ExecutorService`) ensure that the KDC can support up to 10 simultaneous connections. Because of the use of threads, each thread should share the exact same `NonceCache` to prevent replays across connections.

In order for the KDC to know about principles and their secrets, we will use a secret store that is stored on disk as a JSON file. In practice, this file would be protected using file permissions on the hosting server. An example secrets file (`secrets.json`) is attached to the assignment on Classroom. The secrets file is a JSON object that has a single array with key `secrets`. Each entry of the array is an object with an entry that consists of a key `user` which holds the username as a string and an entry with key `secret` which holds the password associated with the user.

The configuration JSON file, `config.json` requires a bit more discussion. It consist of the following fields:

- A string representing the path to the *secrets* file, that is reference by key `secrets-file`. Each secret consists of a user name and their password **in clear text**. This reason for clear text storage is the CHAP authentication protocol (RFC 1994).
- An integer representing the port number for the server to listen on, that is referenced by key `port`.
- A string that represents a value of type `long` that represents the validity period of a ticket (see session key protocol description below), that is referenced by key `validity-period`.



Do not change the names of the keys or type of entries in my JSON files, I will use these for testing your code.

The KDC server will interact with the client using the session key protocol which consists of two phases:

1. An authentication phase that uses CHAP (RFC 1994) together with the preshared secret (found in `secrets.json`) to authenticate the client to the KDC.
2. A session key request phase where the client requests a session-key-ticket pair and the KDC satisfies the request (encrypting the resulting message with the key, derived from the password, shared by the KDC and client).

Echo Service

Like the KDC server the echo service should read all of its configuration information from a JSON configuration file and then uses this information to startup and enter an infinite loop waiting on a new socket connection.

Servers are normally threaded and your server should be as well. Using the thread-pool pattern (`ExecutorService`) ensure that the echo service can support upto 10 simultaneous connections. Because of the use of threads, each thread should share the exact same `NonceCache` to prevent replays across connections.

The configuration JSON file, `config.json` requires a bit more discussion. It consist of the following fields:

- A string representing the the service name, that is referenced by key `service-name`.
- A string representing the service secret (password), that is referenced by key `service-secret`.
- An integer representing the port number for the server to listen on, that is referenced by key `port`.



Do not change the names of the keys or type of entries in my JSON files, I will use these for testing your code.

This service should participate in the service communication protocol. Which consists of a handshake phase and a communication phase. During the communication phase the echo service receives a message, decrypts the payload, converts the decrypted string to all uppercase, sends a new encrypted message containing the transformed string, and closes the connection.

Protocols

In this section we discuss our protocols in detail. I have the following protocol implementation requirements for you:

1. All message should be sent as JSON strings representing objects. You will want to use the `writeSerializedObject` method which is provided by `JsonIO`. The first parameter of `writeSerializedObject` must be a descendant of `merrimackutil.json.types.JSONType` and second must be a `PrintWriter` object. The `writeSerializedObject` method produces a non-formatted form of the JSON that is amenable to communication over `Sockets`. This is a simplifying assumption in production grade protocols it is most common to encode the messages into a binary format, this is an extra layer of complexity we don't need.
2. All messages should have a `type` field that is a short string the represents the type of message you are working with. Again, this is a simplifying assumption to make identifying messages easy.
3. Any time you need to work with a nonce you should use the `NonceCache` class I've provided for you in `merrimackutil.util`.
4. All encryption should be done using `AES/GCM/NoPadding`.
5. All password-based key derivation should be done using `SCRYPT` using with username as the salt.
6. To maintain interoperability, I will ask that your message formats match exactly as described below[1].

[1]Interoperability is a key property of protocols. This basically allows for any program implementing the protocol to talk to any other protocol implementing the protocol. For example, the HTTP protocol allows many different types of web browsers to talk to many different types of web servers.

Session Key Protocol

This is a protocol run by the KDC and Client. The service does not participate. At the conclusion of the protocol the client will be in possession of a ticket it can use to initiate a communication with the service. As mentioned previously, this protocol consists of two phases: the CHAP phase and the session-key request phase.

CHAP (RFC 1994)

We will implement RFC 1994 with a slightly modified message format – since we will be working with JSON messages and RFC 1994 uses a binary format.

Message 1. (Claim) Client sends to server a claim of identity.

Message 2. (Challenge) Server sends the client a random nonce. For our purposes this will be 32 bytes. If the server can't find the clients name in its secret store it should, instead, send a status message to the client indicating a failure.

Message 3. (Response) Client send the KDC the hash of the concatenation of the password and challenge. For our purposes we will hash using SHA2-256 (`SHA256`). The KDC uses information available to it to construct the hash in the same way and compare. If the comparison is good, the client *must* know the shared secret.

Message 4. (Result) KDC sends a status message to the client which is either success or failure.

Session Key Request/Ticket Request

This is a two message addition to the end of the CHAP protocol.

Message 1. (Ticket Request) This message the client sends the KDC the service name and its user name.

Message 2. (Ticket Response) This message the KDC sends the client. It consists of two components a session key k_s encrypted using the root key derived from the clients password and a ticket which consists of:

- The creation time (a result of a call to `System.currentTimeMillis`)
- The validity time (in milliseconds)
- The user name of the client.
- The name of the service.

- The IV used during encryption.
- The encrypted session key k_s. The ciphertext contains the authentication of the creation time, validity time, user name of client, and name of service.

Service Protocol

The service protocol is run between the client and the service. This protocol consists of two phases a handshake phase were the client gives the service a ticket and both parties convince each other they know the session key (similar to Kerberos) and a communication phase where they communicate with each other using the session key. Both parties should maintain an independent nonce cache. Any time a message is received its nonce is checked against the cache if it's found the message is a replay, the protocol should be terminated (i.e. connection closed). If the the nonce is not in the cache it should be added. Finally, any time a nonce must be constructed, the cache should be asked (this will automatically add this nonce to the cache).

Handshake Protocol

The goal of this phase is to have both the client and service convince one another they know the session key k_s.

Message 1. (Client Hello) The client sends a nonce N_c and the ticket to the service. If the ticket is no longer valid, an error message should be sent back to the client.

Message 2. (Handshake Response) The service sends the client a message that contains.

- A fresh nonce N_s
- The service name
- The IV used for encryption
- The encryption of N_c with session key k_s. The ciphertext contains the authentication of the creation time, validity time, id, and service name in the given order.

Message 3. (Client Response) Provided message 2 is valid, the client proves that they know the session key k_s by sending a message that contains:

- A nonce N_r
- The client name
- The encryption IV
- The encryption of N_s using key k_s. The ciphertext contains the authentication of the creation time, validity time, id, and service name in the given order.

Communication Phase

This phase can be as long or as short as necessary. The messages created by the client and service Hello are exactly the same, and are called service messages. The message consists of the following:

- A nonce
- The client user name
- The service name
- The IV used for encryption
- The encryption of a message m with key k_s. The ciphertext contains the authentication of the client user name, service name, and nonce in that order.

If at any time a nonce is repeated the message should be discarded and the channel closed (this is the simplest recovery method, there are others). Please remember that each message should be sent with a unique IV as well.

> **!** The nonce cache from the first phase should be the same cache used in the second phase.

Helpful Hints

I offer the following helpful hints as you embark on the project:

- Networking code is notorious for getting complicated fast. In short, abstraction is your friend. When I write I almost never interact with a socket directly. I generally create a socket wrapper class that represents the entire channel (send and receive) and provide myself with a `sendMessage` and `receiveMessage` method that handle all of the background stream manipulation and either gives me back a new `Message` object or takes one. This also allows me to simplify debugging by adding code to this channel class that prints every message sent and received so I can trace the network behavior (of course this could be done by using wireshark as well). Finally, the last advantage of abstracting away the socket is that you can implement your protocols on top of channels very easily.
- Protocols are beautiful examples of state machine based programming. A *state machine* is a directed graph where each node is called a state and each edge is represented with an event that if that event is performed, allows the machine to transition to a new state. If you have had CSC 3320 Operating Systems, CSC 3555 Computability & Complexity, or CSC 3910 Graphics you have seen state machines. To implement a state machine, just add a variable, `state`, that keeps track of the current state. You should use a `switch` statement to perform the action of the the current state and then go to the next state by updating the `state` variable.

- Only one `ServerSocket` can be bound to a port at a time. This means that the KDC and service should be listening on different ports. Remember these port numbers should be on the high end of the numbers (anything 1024 or below is reserved, and probably used by your machine) It is possible that a port can be hung during development. You can correct this by resetting your network card (or just restarting your computer).
- Use the *localhost* IP address `127.0.0.1` for any host name you use in this application. While your code can work on a LAN, it will add unnecessary complications to development and testing. Once you get your application working on one machine, you are welcome to come see me to borrow a switch and try it out on a LAN you construct in the CaDS lab or out of you and your team mates computers.
- You should *not* have to disable your firewall to work on this project. Again, **do not disable your firewall**.
- I've mentioned it earlier but, **start early** and work as a team.
- In order to get access to the `NonceCache` you will need to use `merrimackutil` The `NonceCache` is found in `merrimackutil.util`.
- In my JSON configuration files you will see an optional entry with key `debug` this is to turn on the debugging features of the service/KDC. If you don't want to have that key in your configuration files you can safely remove it. It's not part of the grade but, may help with debugging your server code without having to recompile. If the value of `debug` is false, no debugging information should be output to the screen.
- For JSON and command line support download the latest release JAR file form `https://github.com/kisselz/merrimackutil`.
- The javadocs for `merrimackutil` are hosted on `https://cs.merrimack.edu/merrimackutil`.
- The javadocs for the JCE available on-line.
- The Java Security Developer's Guide is also useful `https://docs.oracle.com/en/java/javase/22/security/index.html?authuser=0`.
- I will not only be evaluating your use of security practices but, also general software design practice (use of git, general code architecture, etc. Please see the rubric).
- Remember that you should **never** hard code absolute paths into applications.
- An example of a thread pool is provided on classroom in the unit 6 module.

Testing

To test your application I have provided you with necessary files for the kdc in folder `kdc-config` a secrets store file `secrets.json` a server configuration file `config.json` and for the service in folder `service-config` a configuration file `config.json`. Your code should work with my provided files.

Examples

Both my service and kdc daemon have the same basic options:

```
$ java -jar dist/kdcd.jar -h
usage:
  kdcd
  kdcd --config <configfile>
  kdcd --help
options:
  -c, --config Set the config file.
  -h, --help Display the help.
```

and the service

```
$ java -jar dist/echoservice.jar -h
usage:
  echoservice
  echoservice --config <configfile>
  echoservice --help
options:
  -c, --config Set the config file.
  -h, --help Display the help.
```

As an example of running the client application with now arguments

```
$ java -jar dist/kdcclient.jar
usage:
  client --hosts <hostfile> --user <user> --service <service>
  client --user <user> --service <service>
options:
  -h, --hosts Set the hosts file.
  -u, --user The user name.
  -s, --service The name of the service
```

When we run our client program for the echo service

```
java -jar dist/kdcclient.jar --user alice --service echoservice
KDC Password:
Message to send: hello world
HELLO WORLD
```

Observe that the password is never echoed to the screen (thus preventing shoulder surfing attacks).

Project Management

Group projects require the use of good project management strategies. These skills are important to master and thus are practiced across the CS curriculum.

Source Control

In all projects in this class you will use source control. The source control system the department has chosen is `git`. In particular we will use GitHub. For this project please create a *private* GitHub repository under an account linked to your Merrimack e-mail.

As part of your grade on this project I will be requiring good use of the source control. This means your project should have several commits over the course of time.



Don't develop your entire program outside of source control and then add it to source control before submission!

Every commit should have a well worded commit message that explains what the update to the repository is all about. Please be sure to not put automatically generated files in the repository. This includes `class` files, `jar` files, and `zip` files.

Since this a group project, each member's contributions will be judged based on the commit history. For example, a group member that has one tiny commit will receive a lower grade than other members. To make your submission clear, please have a `README` in your repository that lists all of the members of the group.

Project Board & Issue Tracker

Since this is a group project, I'm requiring you to utilize the group management tools integrated into GitHub. There are two main group management tools:

1. The issue tracker which is utilized for creating issues that are directly tied to code (e.g., adding a feature, fixing a bug).

2. The project board. The project board can be used to track both planning of code (it will let you create issues) as well as other project oriented tasks.

For every project you will be *required* to use the project management. I request that you use project board to manage dates and the project schedule. You should create issues for any code that needs to be done (again can be done using the project board). By way of example adding JSON support to an object would be an issue and testing the project would be a task in the project board. Every task should have an assigned owner and date of completion. When working with tasks they will be labeled drafts, this is OK. Only make issues out of actual code issues.

The task workflow is as follows:

1. A task is created, an owner and completion date is assigned. The task is placed in the *ready* queue. If the task is a coding task, an issue is created from the task.

2. The owner begins work on the task, the task is placed in the *in progress* queue.

3. When the owner completes the task, it should go in the done queue.

Your first task as a group is to set up the schedule. This shouldn't take more than 30 minutes to an hour of time. The use of project management tools will help me in determining the grade for each group member.

Being a Good Team Member

Part of developing software in a group is learning to be a good team member. Here are some guidelines:

- When you have agreed to complete a task, work towards completing it by the deadline. If you're struggling to complete it by the deadline, communicate with your team and update the task on the project board.

- Do not rush to finish the project as quickly as possible. Everyone codes at their own pace. If you steal work from a team mate before the internal deadline, I will deduct points from you.

- If you are done with your tasks early, feel free to offer to help other members of your team (this is called *swarming* in industry).

- Do not engage in ad-hominem attacks, discuss code issues only in terms of the code and project milestones.

- Do not fight in the repository. This could for example, be completely reverting a team members commit in a hostile way. If you all agree a commit should be reverted, that is a completely different thing.

- If you are confused about a part of the project description, talk to other members of your team. If you are all still unsure *please discuss the issue with me.*

Submitting

To submit this program, please add me as a contributor to your private GitHub repository, my username is (`kisselz@merrimack.edu`). Since this is a group project, there should only be one repository which each group member contributes to. The names of every team member (and their username, if its not their name) should appear in the `README.md` file.

Rubric

Your grade on this project will be determined as follows:

- Good use of source control and project management tools (*10 points*).
- Code well architected (*10 points*).
- CHAP subprotocol correctly implemented (*20 points*).
- Session key generation subprotocol correctly implemented (*20 points*).
- Handshake subprotocol correctly implemented (*20 points*).
- Communication subprotocol correctly implemented(*5 points*).
- All server applications correctly use configuration files (*5 points*).
- Client application interacts with both servers without user intervention (*5 points*).
- The protocol messages allow for interoperability (*5 points*)

Extra Credit

For **20 points** of extra credit, you can add an additional service to your system. This service should return the *signature* on the SHA2-256 hash of any message it receives. This service should operate using a configuration file similar to the echo service's configuration file. There is one difference, the service should have a signing key field (called `signing-key`) that has a base 64 encoded RSA signing key as its value. In this way every message is signed with exactly the same key. Such signing services are common in software companies where different divisions produce different software packages but, all the packages should be signed with the corporation's key.

The signing service should use the service protocol as described above, since it is an instance of a service.

This service should be given its own entry in the client's `hosts.json` file. The client should also be modified in such a way that when interacting with the signing service it outputs a base 64 encoded string representing the received signature. **The echo service should still work**.